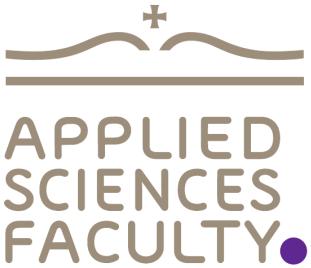
C++ — лекція 12. Класи — 2. Інкапсуляція





©Олег Фаренюк,
Creative Commons - Attribution Share Alike (CC BY-SA)

Когерентність (самоузгодженість) даних

Якщо програміст, помилково, чи навмисне, не до кінця розуміючи, що робить, змінить чисельник чи знаменник дробу, не скоротивши його, результат може стати непередбачуваним! Клас SimpleRatio має ряд *інваріантів*, які при тому буде порушено.

Щоб захистися від того, слід "приховати" дані (члени) класу, заборонивши доступ до них ззовні.

C++ надає спеціальні засоби для цього — модифікатори доступу[1].

[1] Переклад — мій, не вдалося в словниках знайти офіційний. Англійською — access specifiers.

Приклад:

```
class SimpleRatio1 {
public:
    int chys, znam; // numerator, denominator
    void reduce();

        ..............
}
class SimpleRatio2 {
private:
    int chys, znam; // numerator, denominator
    void reduce();
public:
        ..............
}

SimpleRatio1 r1(1,5);
r1.chys=555; // OK
r1.reduce(); // OK

SimpleRatio2 r2(1,5);
// r2.x=555;    // <====== Заборонено!
// r2.reduce(); // <====== Заборонено!
```

Існує три модифікатори доступу:

- **public:** — поле чи метод доступні всім.
- **private:** — поле чи метод доступні тільки методам цього класу.
- **protected:** — поле чи метод доступні методам цього класу та класів, що від нього успадковують (класи-потомки)[2].

[2] Про успадкування детальніше далі.

Однак, заборонивши псувати інваріанти, заборонивши ззовні змінювати чисельник та знаменник, ми втратили можливість звернутися до них, хоча б для читання. Це можна виправити.

```
class SimpleRatio {
private:
    int chys, znam; // numerator, denominator
        .............
public:
    inline int get_znam() const
    {
        return znam;
    }
    inline int get_chys() const
    {
        return chys;
    }
        .............
};
```

В принципі, немає нічого кримінального і в бажанні їх змінювати. Однак, слід робити це так, щоб інваріанти дотримувалися:

```
class SimpleRatio {
private:
    int chys, znam; // numerator, denominator
        .............
public:
    inline SimpleRatio& set_chys(int v)
    {
        chys = v;
        reduce();
        return *this;
    }

    inline SimpleRatio& set_znam(int v)
    {
        znam = v;
        reduce();
        return *this;
    }
        .............
};
```

Дублювання коду — погано!

Тому всі дії по підтримці інваріантів і винесено в окрему функцію, reduce().

- Слід розуміти, що модифікатори доступу, private, protected, public — лише засіб захисту від помилки. Його легко обійти, взявши адресу, здійснивши приведення типів, і т.д. Як то кажуть, захист від дурня не допомагає від винахідливого дурня.
- Різниця між class і struct в тому, що по замовчуванню class використовує private, struct — public:

struct X {
private:
.....
};
	≡	**class** X {
		.....
		};

class X {
public:
.....
};
	≡	**struct** X {
		.....
		};

Фізичне розташування полів

- В C++98, якщо в класі трапляється декілька специфікаторів доступу:

```
class some_t{
private:
    int a;
    int b;
private:
    int c;
public:
    int d;
private:
    int e;
};
```

  компілятор має право перевпорядковувати їх в фізичному представленні класу. Гарантується лише, що відносний порядок в межах одного блоку збережеться – a завжди буде перед b. Але компілятор має право поставити c чи d *між* a і b – хоча, здається, жодна реалізація цього не робила.
- C++11 заборонив змінювати порядок для одного виду специфікатора – e завжди має бути після b.
- В чернетці C++23 заборонили переставляти всі поля, принаймні – в межах одного класу. Предкові дані можуть йти як до, так і після даних нащадка.
- Детальніше див.: `https://quuxplusone.github.io/blog/2022/03/04/non-standard-layout-guarantees/`.

using-оголошення

- Клас не може зробити видимим те, що він сам не бачить, однак може надати доступ іншим для доступного йому:

```
class B{
private:
    int private_field;
protected:
    int protected_field;
public:
    int public_field;
};

class D: public B{
public:
    // using B::private_field; //  <=== Заборонено
    using B::protected_field;  // OK, доступно користувачам D.
};
```

Інкапсуляція — приховування інформації.

Клас надає користувачам **інтерфейс** — єдиний спосіб працювати із ним. В нашому прикладі це — конструктор, set_chys(), set_znam(), get_chys(), get_znam(), математичні методи, методи порівняння, тощо.

Деталі реалізації — такі як конкретні способи збереження чисельника і знаменника, методи скорочення та виконання математичних операцій, тощо, приховані. Програміст, що користується класом і не повинен про них знати! Зокрема, деталі реалізації можна змінювати, не змінюючи програм, які користуються класом.

Інкапсуляція — перше із трьох найважливіших понять об'єктно-орієнтованого програмування.

Функції-помічники

Є такі корисні функції, які працюють із об'єктами якихось класів, доповнюючи можливості, що надаються методами, однак, не потребують доступу до приватних даних класу. Часто більш правильно їх робити незалежними функціями, а не методами.

```
bool check_if_equal(const SimpleRatio &a,
                    const SimpleRatio &b);
```

Друзі класів

Описавши правило приховування — private дані та методи, зразу перейдемо до того, як його можна порушити. Є можливість зробити виняток — вказати класи та функції, які таки матимуть право доступу до приватної області класу. Такі функції та класи називаються друзями.

Всі друзі задаються під час створення класу, тому вони — не таке сильне порушення інкапсуляції, як може здаватися. Однак, дружба — найсильніший взаємозв'язок між класами чи між класом і функцією. Тому її слід використовувати обережно!

Функції-друзі

Щоб вказати, що якась функція є другом, слід **в описі класу** задати прототип функції, перед яким знаходитиметься ключове слово `friend`:

```
class TOurRectangle{
private:
    int width, height;
      ..........
public:
      .........
    friend bool Compare(TOurRectangle a, TOurRectangle b);
      .........
};
  ..............
bool Compare(TOurRectangle a, TOurRectangle b)
{
    return a.width==b.width && a.height==b.height;
}
```

УВАГА: хоча функція оголошується в тілі класу, вона НЕ Є його членом/методом. Це просто вказання, що вона є другом.

Класи-друзі

Другом може бути також інший клас.

```
class TOurSquare;

class TOurRectangle{
        .....
      void from_square (TOurSquare a);
};
class TOurSquare {
private:
     int side;
public:
        .....
     friend class CRectangle;
        .....
};

void from_square (TOurSquare a)
{
     width = a.side;
     height = a.side;
}
```

Класи-друзі — 2

- Дружба — одностороння. Якщо клас A є другом класу B, це не означає, що клас B буде другом класу A. Якщо і така дружба потрібна, слід вказати її явно, в описі класу A.
- Дружба не транзитивна — друг друга не є другом, якщо це не вказано явно.
- Друзі можна оголошувати як у закритій, так і у відкритій частині класу – це ні на що не впливає.
- Друзі є частиною інтерфейсу класу.

Приховані друзі (hidden friends)

- Друг може бути оголошеним, або й визначеним, зразу в оголошенні класу:

```
class TOurRectangle{
    .........
  friend bool compare(const TOurRectangle& lhs,
                      const TOurRectangle& rhs); // Definition
  friend int  condition(const TOurRectangle& a){ ..... }; // Declarat
};
```

- Такі друзі можна назвати прихованими. Якщо їх не оголошено окремо, глобально чи у відповідному просторі імен:

```
bool compare(const TOurRectangle& lhs, const TOurRectangle& rhs);
```

  ця функція не братиме участь в пошуку імен, крім як під час Argument-Dependent Lookup (ADL).
- Перевага такого підходу – уникнення неявних перетворень, за допомогою користувацьких операцій перетворення чи конструкторів із одним аргументом, при виклику таких функцій – якщо ви про це не попросили явно.
- Детальніше див, наприклад: `https://www.justsoftwaresolutions.co.uk/cplusplus/hidden-friends.html`
- Ще приклад: `https://quuxplusone.github.io/blog/2021/10/22/`

Пошук друзів здійснюється:

- У області видимості, яка охоплює оголошення відповідного класу, іншими словами — поточній. В області видимості вищих рівнів не заглядається!
- У просторах імен, де оголошено типи аргументів функції-друга.

- "Class invariants" – про інваріанти класів та інкапсуляцію.
- "The 'OO' Antipattern" – хороша стаття про зловживання ООП.
- "A foliage of folly" – спосіб "законно" доступитися до приватних членів. Потрібен лише в екзотичних випадках – автоматизованого тестування, дебагу тощо, уникайте в "індустріальному" коді.
- "Declaring a friendship to self" – ще один експеримент заради експерименту.
- "Value semantics" – есей на тему.

